Energy-Efficient Motion Planning for Autonomous Vehicles Using Uppaal Stratego

Naeem, Muhammad; Gu, Rong; Seceleanu, Cristina; Guldstrand Larsen, Kim; Nielsen, Brian; Albano, Michele

Published in:
Theoretical Aspects of Software Engineering

DOI (link to publication from Publisher):
10.1007/978-3-031-64626-3_21

Publication date:
2024

Document Version
Accepted author manuscript, peer reviewed version

Energy-Optimized Motion Planning for Autonomous Vehicles Using UPPAAL STRATEGO *

Muhammad Naeem[1], Rong Gu[2], Cristina Seceleanu[2], Kim Guldstrand Larsen[1], Brian Nielsen[1], and Michele Albano[1]

[1] Department of Computer Science, Aalborg University, Aalborg, Denmark
{mnaeem, kgl, bnielsen, mialb}@cs.aau.dk
[2] Division of Networked and Embedded Systems, Mälardalen University, Sweden
{rong.gu, cristina.seceleanu}@mdu.se

Abstract. Energy-efficient motion planning for autonomous battery-powered vehicles is crucial to avoid frequent battery recharge. This paper proposes algorithms for synthesizing energy- and time-efficient motion plans for battery-powered autonomous construction equipment. We use priced timed games to model an appropriate abstraction of the autonomous vehicle and the environment. Based on the model, we synthesize energy- and time-optimal motion plans by using Q-learning in UPPAAL STRATEGO. Via experiments, we show that pure Q-learning is insufficient when the problem becomes complex, e.g., motion planning in large environments. To address this issue, we introduce Concatenated Motion Planning (CoMOP), which divides the environment into several regions, synthesizes a motion plan in each of the regions, and concatenates them into an entire motion plan for the whole environment. CoMOP enhances the applicability of Q-learning to large and complex environments, reduces the synthesis time, and provides efficient navigation and precise motion plans. We conduct experiments with our approaches in an industrial use case. The results show that CoMOP outperforms simple Motion Planning regarding synthesis time and the ability to deal with complex models. Moreover, we compare the energy- and time-optimal strategies and visualize their differences on different terrains.

Keywords: Motion planning · UPPAAL Stratego · Q-learning

1 Introduction

Autonomous vehicles are equipped with advanced sensors, cameras, and communication devices, which enable them to perceive the environment and perform various tasks autonomously. In particular, the use of autonomous vehicles in construction sites can lead to increased productivity, safety, and reduced costs. One

* The research was supported by the ERC Advanced Grant LASSO and Villum Investigator Grant S4OS, and the Swedish Knowledge Foundation via the synergy project ACICS – Assured Cloud Platforms for Industrial Cyber-Physical Systems, grant nr. 20190038, and the profile project DPAC - Dependable Platform for Autonomous Systems and Control, grant nr: 20150022.

of the key challenges in the design and operation of autonomous vehicles is to ensure their energy efficiency. This is particularly important for battery-powered vehicles, where the limited energy capacity of the battery constrains the vehicle's operating range and duration. Therefore, an energy-optimal motion-planning algorithm is crucial for the deployment of autonomous vehicles in real-world scenarios. However, motion planning with energy optimization is not trivial. Intuitively, a time-optimal motion plan should be energy-optimal, which means a path-finding algorithm like A* [16] is sufficient. However, our experiments show that these two kinds of motion plans are not necessarily the same on different terrains. For instance, when slopes exist, the fastest motion plan can be very energy-consuming, but traveling in a curve might cost less energy.

In recent years, reinforcement learning [17] has become popular for solving the motion-planning problem [1][3]. The idea is simple and attractive: let the machine run by itself and accumulate its actions' reward gathered from the environment. Given enough episodes of learning, the machine would solve the problem automatically without human intervention. However, the reality is that reinforcement learning algorithms are extremely data-hungry, and tuning a reward function can be very difficult and time-consuming, especially when multiple factors are considered at the same time. For example, according to our experiments, synthesizing an energy-optimal motion plan by using Q-learning [18] in a 20×20 environment with 2 obstacles costs almost 2 hours. This motivates us to develop a new method that is able to solve real-world motion-planning problems in a reasonable time and provide a systematic way of evaluating the learning results, such as the probability of finishing a job before the energy consumption exceeds a threshold.

In this paper, we present a study on the development of *energy-optimal motion planning* for battery-powered autonomous vehicles. First, we model the autonomous vehicles and the environment as *priced timed games* (PTG) [5]. By using this modeling language, we can construct the control logic as timed games (TG) and encode the vehicle's nonlinear dynamics and energy consumption as ordinary differential equations (ODE) in the TG. Next, we design *methods* for solving the PTG by using Q-learning, that is, synthesizing time- and energy-optimal motion plans of the model. We construct the model and conduct the synthesis in UPPAAL Stratego [6], which is a toolset of modeling, simulation, verification, and synthesis of timed games. As aforementioned, motion planning by Q-learning is restricted by the scale of the problem, such as the environment size and obstacle numbers. Therefore, we design a *novel algorithm*, called *concatenation-based motion planning* (CoMOP), which splits the environment into sub-components, synthesizes a sub-motion plan in each of the sub-components, concatenates the sub-motion plans into a complete one, and verifies the complete plan by using statistical model checking. When concatenating two motion plans, we must ensure the *validity* and *automation* of the concatenation, that is, the final set of states of the first motion plan is equivalent to the initial set of states of the second motion plan, and the work must be done automatically by our tool, such that CoMOP does not break the autonomy of the system. To achieve these, we realize

the concatenation as an external library that is invoked by UPPAAL Stratego, in which assertions are used for checking the validity of the concatenation. We evaluate CoMOP in an industrial environment, an autonomous quarry. We design several groups of experiments that have different sizes of quarry and obstacles, and different numbers of obstacles. Via the experiments, we show the profound difference in performance between CoMOP and MOP (motion planning without concatenation), and the difference between time- and energy-optimal trajectories of motion plans on various terrains. In summary, our contributions are as follows:

1. This paper proposes a novel *concatenation-based motion planning* (CoMOP) by reinforcement learning. CoMOP is much more efficient than MOP when the scale of the problem is large.
2. We demonstrate the use of statistical model checking (SMC) in motion planning that uses learning. Via experiments, we show the *qualitative evaluation of the synthesized motion plans* by using SMC, which exposes the correct rate of the resulting motion plan before testing it on an actual system implementation.
3. Our method can generate *time- and energy-optimal motion plans* that are not necessarily the same on different terrains. Our large-scale evaluation in an industrial environment shows the efficiency of obtaining these two kinds of motion plans.

2 Related Work

In recent years, there has been a growing interest in developing energy-optimized motion planning algorithms for autonomous vehicles. Energy consumption can be minimized by a parametric curve fit [13], by applying machine learning [14, 15], or by employing physics-based models of energy consumption [4, 9]. The energy prediction method used by Quann et al. [14, 15] abstracts from kinematic considerations such as velocity and acceleration. Physics-based modeling [9] that accounts for these assumes mostly flat terrains. In Wallace et al. [4], the authors consider elevation and a detailed kinematic model, however, their model is analytical, being validated via simulation. In our work, we model both vehicle kinematics, although not as detailed as in Wallace et al.'s work [4], and consider environments that include slopes, employing a learning-based model-checking technique for synthesizing energy-efficient motion plans automatically.

Compositional reinforcement learning is a promising approach for training policies to perform complex long-horizon tasks. Jothimurugan et al. [11] propose a novel framework based on two reinforcement learning algorithms by formulating the learning problem of choosing what tasks to perform from an existing set, as an adversarial reinforcement learning problem. Although close in spirit, our approach does not focus on robust task choices w.r.t. functionality, but rather w.r.t. optimizing time- and energy usage by combining task learning with assertion-based space decomposition within priced timed games.

3 Preliminaries

In this section, we overview reinforcement learning, priced timed games and Uppaal Stratego, which form the paper's necessary background information.

3.1 Reinforcement Learning

Reinforcement learning (RL) [17] is a machine-learning method that trains a machine's behavior by letting it interact with the environment, collecting scores of the machine's actions at different states, and calculating the state-action pairs' values. *Q-learning* [18] is a classic RL algorithm that uses the Bellman-optimal equation to calculate the values of state-action pairs, shown as follows.

$$q^*(s,a) = \mathbb{E}[R(s,a) + \gamma \max_{a'} q^*(s',a')], \tag{1}$$

where $q^*(s,a)$ represents the expected value of performing action a at state s, $\mathbb{E}$ denotes the expected value function, $R(s,a)$ is the reward returned from the environment by taking a at s, $\gamma \in [0,1]$ is a discounting value indicating how much the future reward is evaluated in the calculation, s' is the next state originating from s by taking a, and $\max_{a'} q^*(s',a')$ is the maximum reward that can be obtained by any possible next state-action pair (s',a').

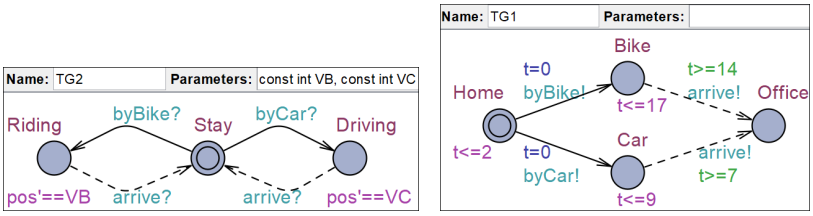
3.2 Uppaal Stratego

Priced Timed Games (PTG) are two-player games played on priced timed automata [2]. We recall the formal definition of PTG as follows. A HA is defined as the following tuple:

$$HA =< L, l_0, X, \Sigma, E, F, I >, \tag{2}$$

where: L is a finite set of *locations*, $l_0 \in L$ is the *initial location*, X is a finite set of continuous variables, $\Sigma = \Sigma_i \uplus \Sigma_o$ is a finite set of actions that are partitioned into inputs (Σ_i) and outputs (Σ_o), E is a finite set of edges of the form (l, g, a, φ, l'), where l and l' are locations, g is a predicate on $\mathbb{R}^X$, $a \in \Sigma$ is an action label, and φ is a binary relation on $\mathbb{R}^X$, $F(l)$ is a delay function for the location $l \in L$, and I assigns an invariant predicate $I(l)$ in/of L, which bounds the delay time in the respective location.

Definition 1. *A Priced Timed Game* $\mathcal{G} =< L, l_0, C, \Sigma, E, Inv >$ *is a tuple, where* L *is a finite set of locations,* l_0 *is the initial location,* C *is a finite set of non-negative real-valued variables named clocks,* $\Sigma = \Sigma_c \cup \Sigma_u$ *is a finite set of actions (*Σ_c *and* Σ_u *denote a set of controllable and uncontrollable actions, respectively),* $E \subseteq L \times \mathcal{B}(C) \times \Sigma \times 2^C \times L$ *is a finite set of edges, where* $\mathcal{B}(C)$ *is the set of guards over* C*, that is, conjunctive formulas of constraints of the form* $c_1 \bowtie n$ *or* $c_1 - c_2 \bowtie n$*, where* $c_1, c_2 \in C$*,* $n \in \mathbb{N}$*,* $\bowtie \in \{<, \leq, =, \geq, >\}$*,* 2^C *is a set of clocks in* C *that are reset on the edge, and* $Inv : L \to \mathcal{B}(C)$ *is a partial function assigning invariants to locations.* □



(a) (b)

Fig. 1: An example of NPTG templates in Uppaal Stratego

Intuitively, *controllable* actions are performed by the system, e.g., autonomous vehicles in this paper, and *uncontrollable actions* are performed by the environment. Uppaal Stratego [6] is a powerful tool for modeling, strategy synthesis, and (statistical) model checking of PTG, in which PTG can be composed in parallel as a *network* of PTG (NPTG) synchronized via *channels*. Fig. 1 depicts PTG templates that can be instantiated into NPTG models by assigning concrete values to the parameters of the template, such as `VB` in TG1. Blue circles are *locations* that are connected by directional *edges*. Double-circled locations are the *initial* locations (e.g. `Stay`). In Uppaal Stratego, there are two special kinds of locations: *urgent* locations (i.e., encircled "u"), and *committed* locations (i.e., encircled "c"). UTG require that time does not elapse in those two kinds of locations, and committed locations are even stricter, that is, the next edge to be traversed must start from one of them. On the edges, there are assignments resetting clocks (e.g., `t=0`) and updating data variables, guards (e.g., `t>=14`), and synchronization channels (e.g., `arrive!` and `arrive?`). On location `Bike` in Fig. 1b, an invariant `t<=17` means that clock `t` must never exceed 17. Derivatives of clocks can be specified by ordinary differential equations (ODE), such as `pos'==VB` in Fig. 1a.

A solution of an NPTG is a *winning strategy*, which shows the "system" player what controllable actions to choose at each location, such that it can eventually reach the desired goal. For example, Fig. 1 models a driver taking different transportation tools to go to his office. In Uppaal Stratego, we can synthesize a strategy that guides the system player (e.g., the driver) to reach its goal (e.g., the office) by using the following query.

```
strategy s = minE(cost){ExpList1}->{ExpList2}:<> goal
```
(3)

where `minE(exp)` means that strategy `s` is to minimize the value of expression `cost`, which is a formula that is referred to as a *cost function* in the machine-learning domain. `ExpList1` and `ExpList2` are lists of expressions that are passed to a learning algorithm, which is Q-learning [18] in this paper. Past studies [10][8] indicate that it is preferable that expressions in `ExpList1` are discrete variables and the ones in `ExpList2` are continuous variables (i.e., clocks). When running this query in Uppaal Stratego, the tool randomly simulates the model and

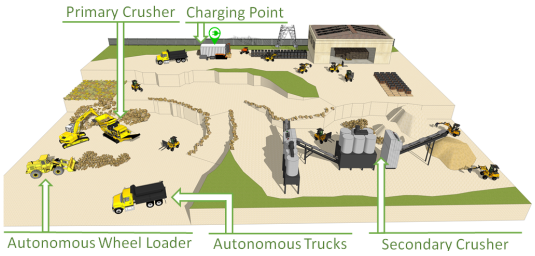


Fig. 2: An example of an autonomous vehicle working environment

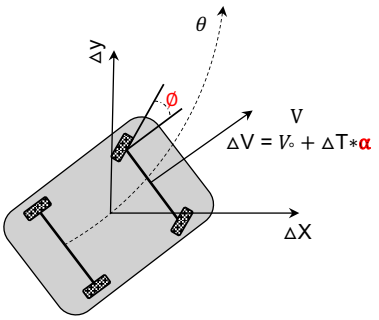
samples its execution traces, which are passed to the learning algorithm. At the end of the query, `<> goal` specifies the criteria for selecting simulation traces, that is, the ones that reach a state where `goal` is satisfied are selected. Note that the learning algorithm only gets controllable actions in those traces, which is the so-called *partial observation* of the state space [8]. Last but not least, the latest version of UPPAAL STRATEGO supports calling external C libraries during the learning, simulation, and verification of models, which is an important feature that is utilized in this work.

4 Use Case and Models

In this paper, we demonstrate the applicability of our method on an industrial case study provided by Volvo Construction Equipment, Sweden. The use case focuses on optimizing the energy consumption of autonomous wheel loaders used for transporting materials within a construction site. An example is shown in Fig 2, where the autonomous wheel loader needs to transport stones to a primary crusher to crush them into smaller pieces. After that, it transports the crashed stones to the secondary crusher, which is the destination of the stones. We aim to develop energy-optimal motion planning for battery-powered autonomous vehicles to extend the vehicle's operating time on a single charge.

4.1 Vehicle's kinematic Model

In Figure 3, one can see a simple kinematic model that illustrates the configuration of a vehicle using seven parameters: (x, y, ϕ, α, V, θ, T), where x and y model the vehicle's current position in a 2D environment, ϕ the steering angle, α the acceleration, V the velocity, θ the vehicle's orientation, and T the time.



The model allows calculating the evolution of the vehicle's status, as follows:

$$\begin{aligned} \triangle X &= X + V \cdot cos(\theta) \cdot \triangle T \\ \triangle Y &= X + V \cdot sin(\theta) \cdot \triangle T \\ \triangle \theta &= \theta + \triangle T \cdot \phi \\ \triangle V &= V + \triangle T \cdot \alpha \end{aligned} \tag{4}$$

Fig. 3: Kinematic vehicle model

The trajectory of the vehicle must satisfy a reach-avoid requirement, that is, eventually reaching the destination and always avoiding obstacles. It can be seen that by controlling acceleration α and steering angle ϕ, one can determine the vehicle's position. Therefore, our motion planning is about controlling parameters α and ϕ. In Section 4.2, we introduce how energy optimization is considered in our motion planning.

4.2 Energy Model

The tractive power model described by Lascurain et al. [12] and Gao et al. [7] is widely used to calculate the power required by a vehicle powertrain. The tractive power is the amount of energy required to overcome the vehicle's resistance to motion, and it considers various factors that affect the vehicle's performance, including the environment. The following equation can be used to calculate the vehicles' tractive power:

$$W_{tract} = m \cdot V \cdot \frac{dV}{dt} + m \cdot g \cdot C_{rr} \cdot V + m \cdot g \cdot V \cdot sin(\theta) \tag{5}$$

where W_{tract} is the tractive power, m is the total mass of the vehicle, V is the velocity, g is a constant for gravitational acceleration, C_{rr} is a coefficient of rolling resistance, θ is the road grade. Parameters values obtained from the Volvo project are shown in Table 1.

Table 1: Energy Parameters

Name	*Value*
m	$7-22Ton$
C_{rr}	$2-3\%$
θ	12%
W_{acc}	3.75
η_{wh}	0.99
η_{fd}	0.98
η_{mot}	0.88
η_{batt}	0.98

In this study, we use the energy model presented in the literature [12]. The authors propose a model for estimating the electric power output of an electric vehicle (EV), which considers the efficiencies of the electric components, such as the motor and battery, and the related drive-train components, such as the final drive and wheel efficiencies shown in Eq 6. The equations for calculating the driving power and breaking power of an EV are as follows.

$$\begin{aligned} &Driving\ Power : W_{drive} = W_{acc} + \frac{W_{tract}}{\eta_{wh} \cdot \eta_{fd} \cdot \eta_{mot} \cdot \eta_{batt}} \\ &Breaking\ Power : W_{Breaking} = W_{acc} \\ &TotalPower : W = W_{drive} + W_{Breaking} \end{aligned} \tag{6}$$

where W_{acc} is a conventional vehicle's accessory load (kW), η_{wh} is the wheel efficiency, η_{fd} is the final drive efficiency, η_{mot} is the motor efficiency, η_{batt} is the battery efficiency. Table 1 depicts the values of coefficients used in this paper.

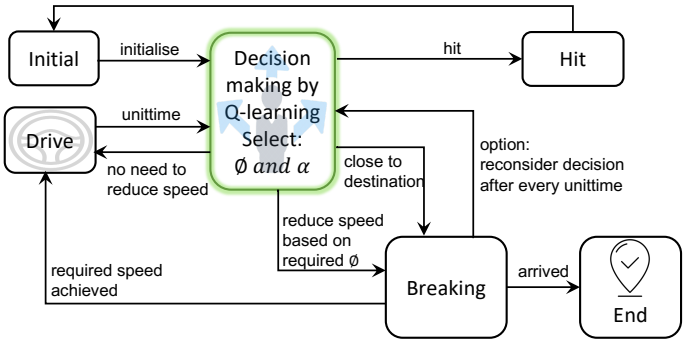


Fig. 4: Vehicle Model in Uppaal Stratego

4.3 Abstract View of Vehicle Model

We develop a framework for simulating a vehicle's movement to find a path to the destination while avoiding obstacles, which includes a dynamic decision-making process based on the Q-learning algorithm. Specifically, we construct a PTG model of an autonomous wheel loader in Uppaal Stratego, which can navigate along the x and y axes. The vehicle's movement is controlled by selecting the right steering angle (ϕ) and acceleration (α), chosen from a strategy synthesized by the Q-learning algorithm. To illustrate our PTG model in an abstract way, Figure 4 depicts a graphical representation of the vehicle model.

The decision-making process of the vehicle is controlled by the Q-learning block, which determines the appropriateness of the chosen ϕ and α values for each time unit during the vehicle's movement, that is, the evolution of x and y. Additionally, when a large ϕ is chosen, and the vehicle's current velocity exceeds a predefined comfortable threshold for that specific ϕ, the model enters the `Breaking` block. In this block, the vehicle can move to the `Drive` block when the velocity is reduced to a required level, or return to its decision-making block for a reconsideration of whether the destination is close. Alternatively, the model also enters the `Breaking` block as it approaches the destination, where it keeps braking until the vehicle fully arrives and stops. Furthermore, if the vehicle hits an obstacle or crosses a boundary of the environment, it enters the `Hit` block to indicate a collision or boundary violation.

Now, we define the problem of motion planning as follows:

Definition 2 (Motion Planning). *Given an autonomous vehicle whose kinematics and energy model are in the form of equations (4) and (6), respectively, a motion plan for the vehicle is a function* $(\theta, v) = f(x, y, t)$*, where* θ, v, x*, and* y *are the state variables of the vehicle in equation (4),* $t \in [0, T]$ *is a time point within* 0 *and* T*, and* $T \in \mathbb{R}^+$ *is the time of reaching the destination. The goal of motion planning is to synthesize motion plans that are time-optimal, i.e.,* T *being minimal, or energy-optimal, i.e., total power* W *being minimal.* □

5 Vehicle Model In Uppaal Stratego

This section demonstrates how we use Uppaal Stratego to model the vehicle's actions. Figure 5 depicts a PTG model that implements the abstract vehicle model in Figure 4. Stratego handles this motion planning problem as a PTG. The model contains two types of components (Locations and Edges), and the edges are further divided into controllable (solid line) and uncontrollable ones (dotted line). The locations of the model represent the system's processes like *Breaking* and *Driving*, and edges are the actions controlling these processes. Table 2 presents some important variables used in the model.

Locations `Breaking` and `Drive` are two important locations in our model. All other locations are used to help in taking different actions between or on these locations. The edge associated with the `Initial` location invokes the

Table 2: Variables used in the model. C, D, I, and B represent clock, double, integer, and boolean variables, respectively

Variable	Type	Description
`X,Y`	C	These variables represent the vehicle's position on x and y axis.
`V`	C	It represents the vehicle's velocity.
`agl`	C	It represents vehicle's orientation labelled as θ in Figure 3.
`W`	C	It is used to calculate the energy consumed by the vehicle.
`ar`	I	It represents steering angle labelled as ϕ in Figure 3.
`VR`	I	It represents acceleration labelled as α in Figure 3.
`vSafe`	D	It controls the velocity under a safe range.

`Initialise()` function to initialize variables such as the initial position and angle. The edge from `T7` is controllable and includes a statement of *selections*, which allows the vehicle model to select an acceleration from a defined range from 0 to 10. Within the `Drive` location, clocks evolve according to the differential equations defined in the model, e.g., `agl'==ar` defines the rotational acceleration of the vehicle.

Now, we describe the movement of the vehicle defined at the `Drive` location. First, the coordinates of the vehicle, x and y, begin to change in the direction specified by `agl` (the orientation of the vehicle) while consuming energy. Second, the loops in the `Drive` location control the value of `agl` between 0 and 360, and restrict the acceleration when it reaches the maximum safe speed (`vSafe`). Third, the model should leave the `Drive` location and move to `T3` when the `x` or `y` value evolves a step on the map. The duration spent in a location is determined by the location's invariants and the guard on the outgoing edge. Therefore, the invariants in location `Drive`, such as `x-xp`$\leq$`UNIT`, and the guard `onestep()` regulate the duration in `Drive` to be *one step*. The length of the step is a constant defined in the model.

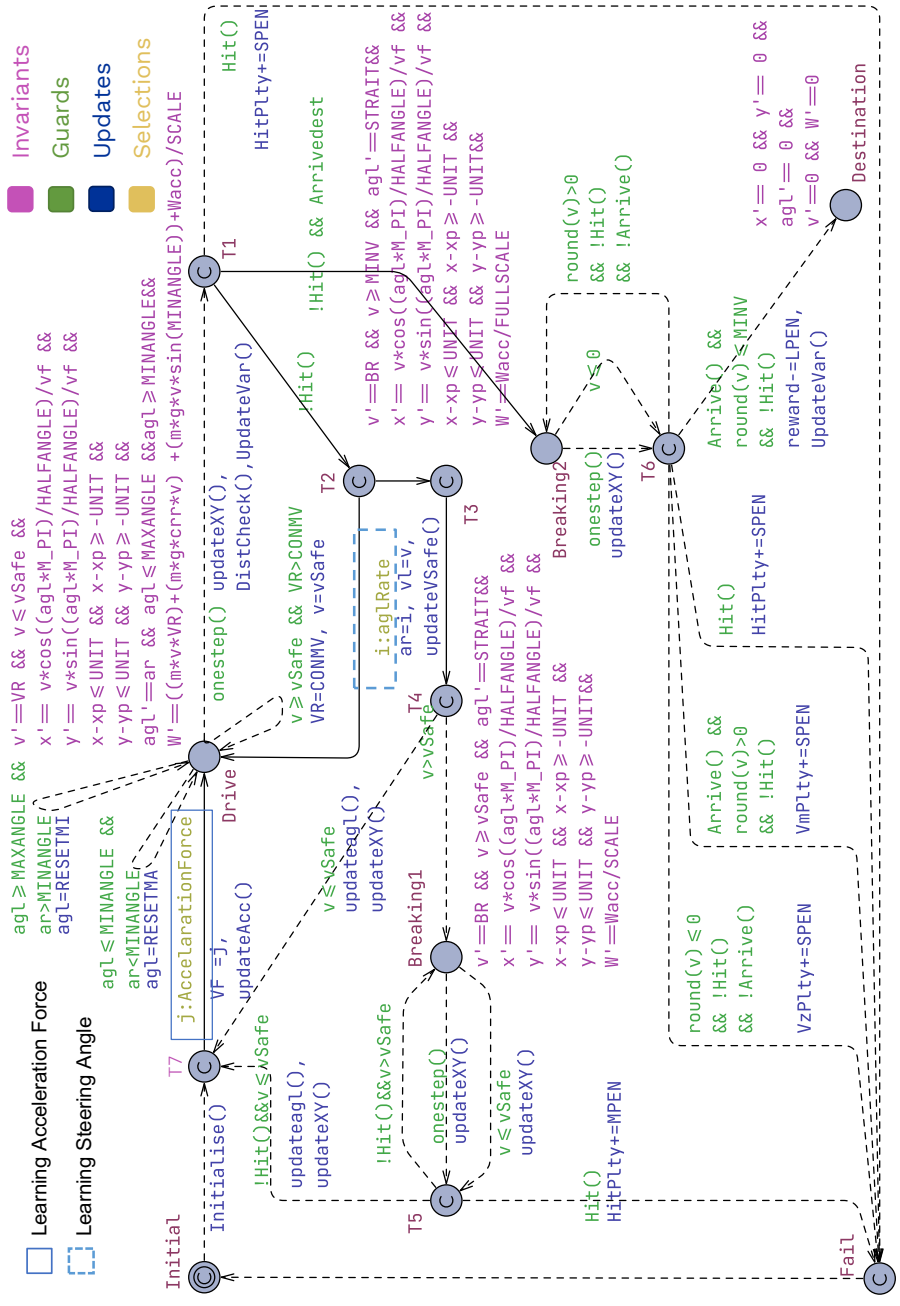


Fig. 5: Vehicle's PTG model in Uppaal Stratego

The model moves from location `T1` to `Fail`, if an obstacle or boundary is encountered. Otherwise, it offers two controllable actions to choose from: continuing to drive or moving to the `Breaking2` location to check if the destination has been reached. The edge from `T1` to `Breaking2` is guarded by the `ArriveDest` variable, enabling it only when the vehicle is close to the destination, thus avoiding unnecessary execution towards the final braking.

On the path toward the `Drive` location, there are two controllable actions: changing the steering angle or moving to the `Drive` location with the same steering angle. On route to the `T3` location, another controllable edge includes a statement of selections to choose a steering angle value ranging from -45 to 45. In this model, we assume that changing the steering angle takes no time. When updating the steering angle, the maximum safe speed also needs to be adjusted based on the new steering angle. If the current velocity exceeds the new maximum safe velocity, the model should transition to the `Braking1` location to reduce the speed before moving to the `Drive` location. Otherwise, it can directly proceed to the `Drive` location.

```
D'==(C1*time+ C2*W+
HitPlty+ VzPlty+
VmPlty)* reward
```

Fig. 6: Optimization function

Penalties and rewards are used in our model to optimize the training process. The model imposes penalties such as `HitPlty` for reaching the `hit` location, `VmPlty` for arriving at the destination with a non-zero vehicle speed, and `VzPlty` for stopping without arriving at the destination. In contrast, a reward is given for successfully arriving at the destination. The model learns to minimize penalties and maximize rewards by incorporating these feedback mechanisms and developing effective motion-planning strategies.

The automaton shown in Figure 6 is used to model the cost function. `D` is a clock variable, and its growth depends on a set of variables (`Time, W, HitPlty , VzPlty, VmPlty, and reward`). These variables are used as a directive of the cast function. The variable `time` is used for the time-optimal strategy, and `W` is used for the energy-optimal one. `C1` and `C2` are coefficients for balancing `time` and `W` in developing strategies. Variable `reward` is updated with a negative value when it reaches the goal and starts reducing the cost function. Uppaal Stratego should minimize this cost function to generate an efficient strategy.

5.1 Strategy Synthesis

This section focuses on the strategy queries employed for motion-plan synthesis. The basic structure of the query is described in section 3.2. Equation 7 is the query for strategy synthesis. Here, `D` is the value of the cost function, and `minE` means the goal of synthesis is to minimize `D`. The query passes the set of discrete and continuous variables (described in section 3.2) to the Q-learning algorithm embedded in Uppaal Stratego to develop a strategy.

$$\texttt{strategy GoFast} = \texttt{minE(D)}\,[\texttt{time} \leq \texttt{T}]\,\{\texttt{location, ar, VR, VzPlty, VmPlty , HitPlty, reward}\} \rightarrow \{\texttt{agl, x, y, v, W}\} : <> \texttt{ptg.end} \,\&\&\, \texttt{time} <= \texttt{T} \quad (7)$$

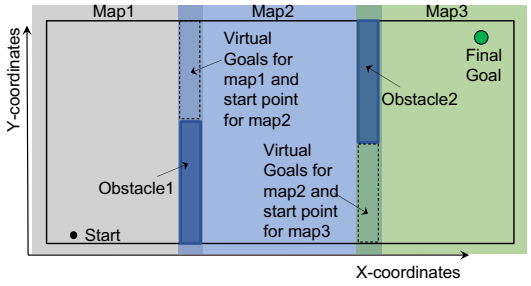


Fig. 7: Abstract view of a map and sub-maps

$$\texttt{simulate}\ [\leq \texttt{T}; \texttt{1}]\{\texttt{x}, \texttt{y}\}\ \texttt{under}\ \texttt{GoFast} \tag{8}$$

After constructing the strategy, it can be analyzed through the statistical model-checking approach. We use query 8 to simulate the model under the control of the synthesized strategy, and the query prints the values of variables `x` and `y`, which are used to depict the moving trajectory of the vehicle. We also use this query to assess other trajectories of variables formed by simulating the model.

6 Assertion-Based Concatenated Motion Planning

We implement the vehicle model in Uppaal Stratego. Next, we use two methods to synthesize motion plans (a.k.a., strategies in Uppaal Stratego) for moving to the destination safely and energy-optimally.
Method 1: MOtion Planning (MOP). MOP considers the entire environment (a.k.a., map) as a single entity and learns a holistic motion plan from the starting point to the destination, while taking into account time or energy consumption.
Method 2: Concatenated MOtion Planning (CoMOP). CoMOP's goal is also to synthesize time- or energy-optimal motion plans. The difference is that CoMOP divides the map into small sub-maps. Each sub-map represents a distinct area of the environment. Figure 7 depicts the subdivision of a map into sub-maps. As shown in the figure, an overlapped region exists between two sub-maps, and these regions serve as a virtual destination for one sub-map and the starting region for the next.

To initiate the synthesis of a motion-planning strategy, we employ a **reverse calculation** approach, starting from *Map3*, where the final goal is located. The objective is to determine the optimal starting point within the overlapped region, which allows us to establish a precise virtual goal point. Furthermore, we analyze the suitable vehicle's orientation required for the forward path strategy in *Map3*. For *Map2*, a similar approach is employed. However, in this case, we also ensure that the model concludes at the starting points previously determined in *Map3*. This sequential process allows us to establish a motion planning strategy that considers both maps and incorporates consistent starting points and orientations.

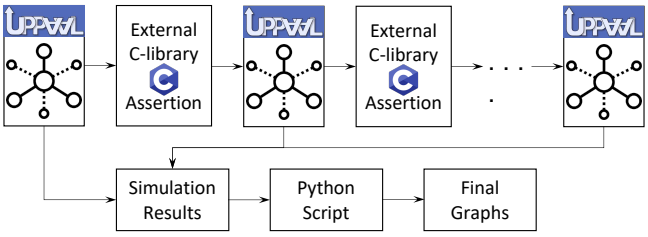


Fig. 8: Cascaded execution of model for CoMOP simulation

In each of the sub-maps, we run forward motion planning, where we use a sequential execution approach for the sub-maps. As illustrated in Figure 8, we first develop a strategy for a sub-map. Subsequently, under the control of the developed strategy, we simulate the vehicle model and obtain the trajectories of variables (such as x, y, agl, W, V, and T, as described in Section 5). The final values of those variables are then forwarded to the following strategy using an external C library, ensuring a continuous flow of information for consecutive sub-maps. We use **assertions** in the external C library to ensure smooth transitions between the **concatenated** strategies, that is, the final values of variables in a sub-map are within the valid regions of variables for the following sub-map.

7 Experimental Evaluation

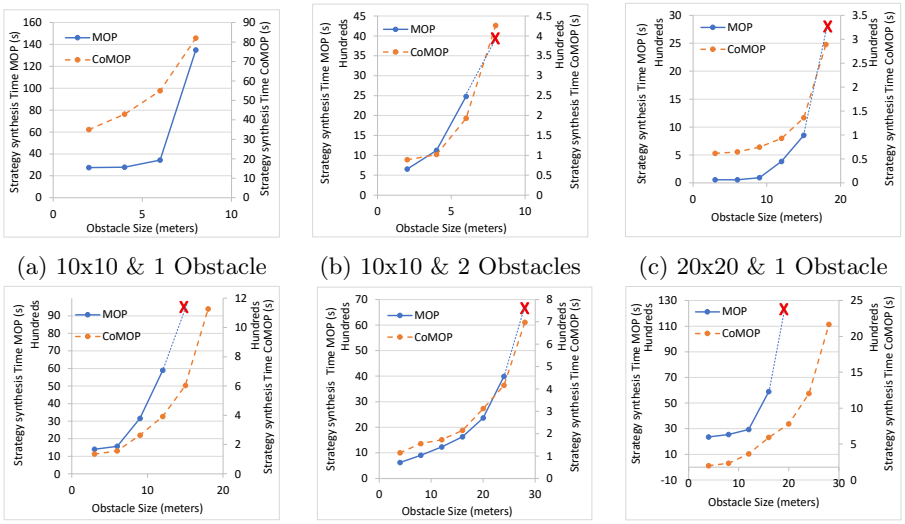
In this section, we introduce the experiments that we have conducted to evaluate the performance of our method. Next, we explore the following research questions and answer them according to the experimental results.

- **RQ1**: Considering map size, slope, obstacle size, and obstacle number, what is their impact on the performance of our synthesis methods?
- **RQ2**: Does CoMOP outperform MOP?
- **RQ3**: Is the time-optimal motion plan significantly different from the energy-optimal motion plan of the same problem?

7.1 RQ1: Impact Factors on the Synthesis Methods' Performance

Based on the industrial use case, we explore the influential factors in this series of experiments by systematically varying three key parameters: map size, number of obstacles, and obstacle size. The experimental design encompasses three different map sizes, namely 10, 20, and 30 units, with each map configuration simulated under two obstacle conditions: one obstacle and two obstacles. Additionally, we examine the effect of obstacle size. Note that the unit of a map is not specified because one step in a map (i.e., the guard `onestep()` in Fig.5) represents the granularity of decision making.

The obtained results, as depicted in Figure 9, provide valuable insights into the impact of these dominating factors. The figure presents the learning time



(a) 10x10 & 1 Obstacle (b) 10x10 & 2 Obstacles (c) 20x20 & 1 Obstacle

(d) 20x20 & 2 Obstacles (e) 30x30 & 1 Obstacle (f) 30x30 & 2 Obstacles

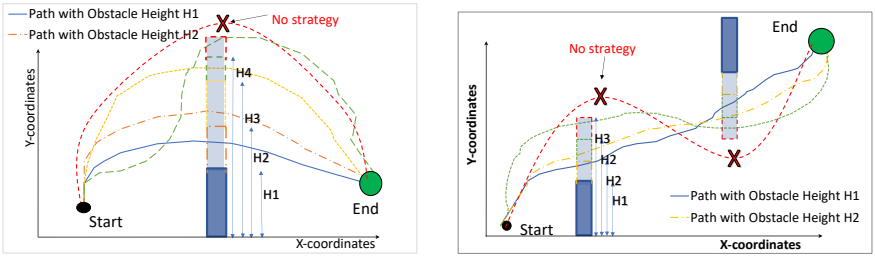
Fig. 9: Strategy Synthesis time with different map sizes, obstacle heights, and number of obstacles

required to synthesize a motion plan for each combination of experiments. By analyzing the results, we can discern the influence of each factor on the simulation outcomes. Increasing the complexity in terms of obstacle size, map size, and the number of obstacles generally resulted in longer simulation times for both the MOP and CoMOP models. Comparatively, CoMOP needs less time to find a strategy, except in the experiment shown in Figure 9a, having a small map with a small obstacle. However, it is essential to note that in certain experiments, the MOP algorithm encounters difficulties in finding a strategy. For example, in the experiment conducted on a 20-size map with a single obstacle of size 18, MOP fails to generate any result (see the red cross in Fig. 9c). CoMOP, on the other hand, demonstrates its ability to address complex motion-planning problems by employing the sub-map division approach.

7.2 RQ2: Performance Improvement of Learning with CoMOP

Figure 10 shows a subset of the experiments of Figure 9, highlighting the impact of obstacle height and the number of obstacles on path trajectories. Increasing obstacle height complicates strategy synthesis, particularly for two obstacles; learning a second sharp turn to reach the goal is more challenging. In Fig. 10b, although the margins between the obstacles to the map edges are quite large, MOP fails to generate any result.

We conduct simulations on a 20-size map with two obstacles to investigate the accuracy of the CoMOP compared to the standalone MOP. CoMOP combines multiple strategies generated by MOP into a holistic strategy for the entire map.



(a) 1 obstacle (b) 2 obstacles

Fig. 10: Path trajectories generated by MOP

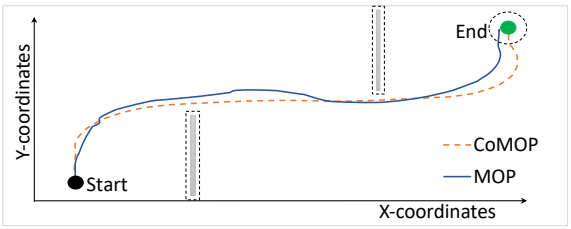


Fig. 11: Comparison of successful strategies developed by MOP and CoMOP

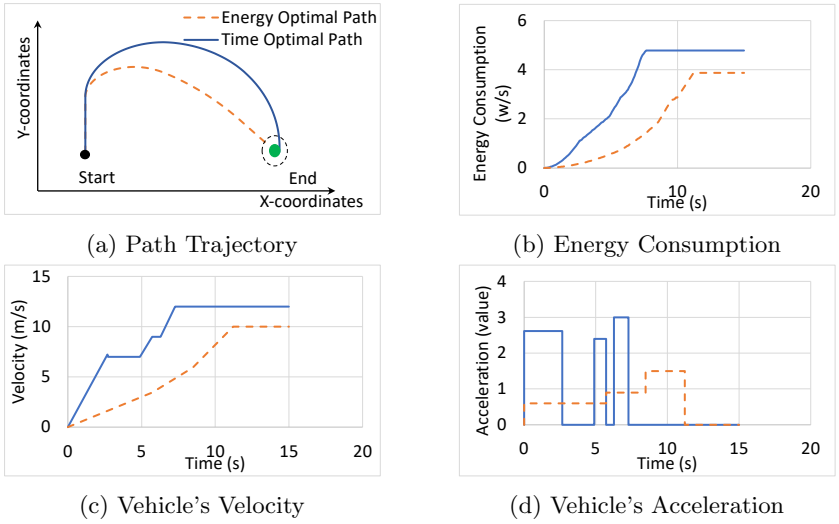
Figure 11 depicts the path trajectories of executing motion plans. It can be observed that CoMOP produces smoother trajectories toward the destination with less dramatic turnings compared to the trajectories generated by MOP alone. In summary, CoMOP improves overall motion-planning performance. It can even solve problems that cannot be addressed by MOP alone.

7.3 RQ3: Time-Optimal and Energy-Optimal Motion Plans

We synthesize energy- and time-optimal strategies and generate path trajectories to the destination by simulating the strategies. In Figure 12a, it can be seen that there are distinct trajectories for time and energy. Figure 12b illustrates that the time-optimal strategy is fast but consumes more energy than that of the energy-optimal strategy. Figures 12c and 12d show that the energy-optimal strategy involves choosing a lower acceleration value and driving at a low speed to conserve energy. Driving at a low speed allows the vehicle to take sharp turns for energy-optimal strategy comparatively, which can be seen in figure 12a.

We further run an experiment in an environment having an obstacle and a 12% slope on the x-axis. Figure 13a depicts that CoMOP develops different trajectories for time and energy, and Figure 13b represents the cost of energy and time for both strategies.

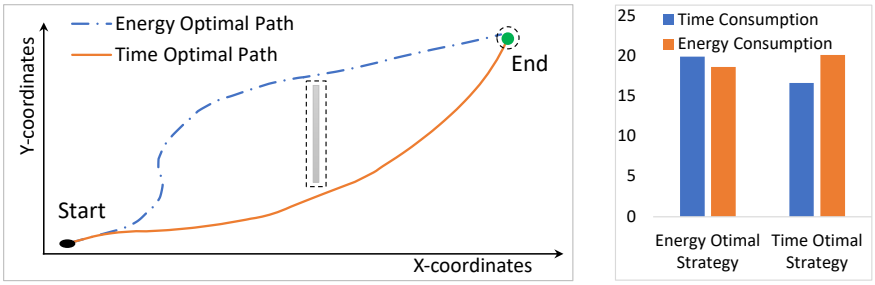
By considering time and energy factors in strategy synthesis, we can generate balanced trajectories that optimize both time utilization and energy usage, by



(a) Path Trajectory (b) Energy Consumption

(c) Vehicle's Velocity (d) Vehicle's Acceleration

Fig. 12: Energy- and time-optimal path trajectories and behavior of velocity, Acceleration, and Energy consumption



(a) Path trajectories with 12% slope on the x-axis. (b) Energy and time

Fig. 13: Syntheses of time- and energy-optimal strategies to reach the destination by avoiding the obstacle.

controlling the constant factors (C1 and C2 shown in figure 6) in the optimization function.

8 Conclusions

In this paper, we propose automatic synthesis strategies for time-efficient and energy-efficient motion planning for autonomous vehicles, using priced timed games in Uppaal Stratego. We present two techniques for motion planning based on the model: MOtion Planning (MOP) and Concatenated MOtion Planning (CoMOP). For CoMOP, we develop an assertion-based method to execute

the Uppaal Stratego model in a cascaded fashion. The results show that the CoMOP performs better than MOP for complex models.

We demonstrate the method's applicability on an industrial use case: energy-efficient path planning for an autonomous vehicle provided by Volvo CE, Sweden. The results show that our method can synthesize time-efficient and energy-efficient path trajectories.

In future work, we plan to extend the model to manage multiple operational vehicles in the environment and concurrent execution of each map sub-part.

References

1. Aradi, S.: Survey of deep reinforcement learning for motion planning of autonomous vehicles. IEEE Transactions on Intelligent Transportation Systems **23**(2), 740–759 (2020)
2. Behrmann, G., Fehnker, A., Hune, T., Larsen, K., Pettersson, P., Romijn, J., Vaandrager, F.: Minimum-cost Reachability for Priced Timed Automata. In: Hybrid Systems: Computation and Control: 4th International Workshop, HSCC 2001 Rome, Italy, March 28–30, 2001 Proceedings 4. pp. 147–161. Springer (2001)
3. Bouton, M., Karlsson, J., Nakhaei, A., Fujimura, K., Kochenderfer, M.J., Tumova, J.: Reinforcement learning with probabilistic guarantees for autonomous driving. arXiv preprint arXiv:1904.07189 (2019)
4. D. Wallace, N., Kong, H., J. Hill, A., Sukkarieh, S.: Energy Aware Mission Planning for WMRs on Uneven Terrains. IFAC-PapersOnLine **52**(30), 149–154 (2019), 6th IFAC Conference on Sensing, Control and Automation Technologies for Agriculture AGRICONTROL 2019
5. David, A., Jensen, P.G., Larsen, K.G., Legay, A., Lime, D., Sørensen, M.G., Taankvist, J.H.: On time with minimal expected cost! In: International Symposium on Automated Technology for Verification and Analysis. pp. 129–145. Springer (2014)
6. David, A., Jensen, P.G., Larsen, K.G., Mikučionis, M., Taankvist, J.H.: Uppaal Stratego. In: Tools and Algorithms for the Construction and Analysis of Systems: 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015, Proceedings 21. pp. 206–211. Springer (2015)
7. Gao, Z., Lin, Z., LaClair, T.J., Liu, C., Li, J.M., Birky, A.K., Ward, J.: Battery capacity and recharging needs for electric buses in city transit service. Energy **122**, 588–600 (2017)
8. Gu, R., Jensen, P.G., Seceleanu, C., Enoiu, E., Lundqvist, K.: Correctness-guaranteed Strategy Synthesis and Compression for Multi-agent Autonomous Systems. Science of Computer Programming **224**, 102894 (2022)
9. Henkel, C., Bubeck, A., Xu, W.: Energy Efficient Dynamic Window Approach for Local Path Planning in Mobile Service Robotics. IFAC-PapersOnLine **49**(15), 32–37 (2016), 9th IFAC Symposium on Intelligent Autonomous Vehicles IAV 2016
10. Jensen, P.G., Larsen, K.G., Mikučionis, M.: Playing Wordle with Uppaal Stratego. In: A Journey from Process Algebra via Timed Automata to Model Learning: Essays Dedicated to Frits Vaandrager on the Occasion of His 60th Birthday, pp. 283–305. Springer (2022)

11. Jothimurugan, K., Hsu, S., Bastani, O., Alur, R.: Robust Subtask Learning for Compositional Generalization. In: 40th International Conference on Machine Learning (ICML2023) (2023)
12. Lascurain, M.B., Franzese, O., Capps, G., Siekmann, A., Thomas, N., LaClair, T., Barker, A., Knee, H.: Medium truck duty cycle data from real-world driving environments: project final report. ORNL/TM-2012/240. Oak Ridge, TN: Oak Ridge National Laboratory (2012)
13. Mei, Y., Lu, Y.H., Hu, Y., Lee, C.: Energy-efficient motion planning for mobile robots. In: IEEE International Conference on Robotics and Automation, ICRA '04. vol. 5, pp. 4344–4349 Vol.5. IEEE (2004)
14. Quann, M., Ojeda, L., Smith, W., Rizzo, D., Castanier, M., Barton, K.: Chance constrained reachability in environments with spatially varying energy costs. Robotics and Autonomous Systems **119**, 1–12 (2019)
15. Quann, M., Ojeda, L., Smith, W., Rizzo, D., Castanier, M., Barton, K.: Power Prediction for Heterogeneous Ground Robots Through Spatial Mapping and Sharing of Terrain Data. IEEE Robotics and Automation Letters **5**(2), 1579–1586 (2020)
16. Rabin, S.: A* aesthetic optimizations, in: Game programming gems. Charles River Media (2000)
17. Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. MIT press (2018)
18. Watkins, C.J.C.H.: Learning from delayed rewards. King's College, Cambridge United Kingdom (1989)